GEN AI DRIVEN SDLC

The New Software Engineering

Spec-driven development, agentic workflows, and the disciplines that turn AI from a toy into a force multiplier.

Anurag Kumar | March 2026

THE STARTING POINT

You're already doing this.

Every good engineering team writes requirements before coding, reviews at checkpoints, documents decisions, and tests before shipping. What's changed is the cost of NOT formalizing it.

84%

of developers now use AI coding tools

1.7×

more issues in AI PRs without structured workflows

3.6 hrs

saved per week by structured AI users

66%

cite 'almost right' as top AI frustration

The opportunity isn't learning something new — it's formalizing what your best people already do and making it the default.

BUILDING BLOCK 1

Intent-Driven Development

The specification becomes the primary artifact.
Code becomes its expression in a particular language.

The idea

Natural language intent — the WHY and WHAT — becomes the source of truth. Code is the last-mile translation. Debugging means fixing specifications, not hunting through implementation.

You're already doing this when you...

- Write a PRD before development starts
- Define acceptance criteria in Jira tickets
- Write problem statements before solutions

The gap: your intent docs aren't structured for AI consumption. Formalize them and the agent can execute.

BUILDING BLOCK 2

Test-Driven Development in the AI Era

Red

Write a failing test that defines expected behavior

→

Green

AI agent writes the minimum code to make it pass

→

Refactor

Clean up with confidence — tests catch regressions

You're already doing this when you...

Define "done" criteria before sprint work begins. Write QA test cases before features ship. Require passing CI before merge. With AI agents, tests become the contract that keeps the machine honest. The agent writes code; tests validate it.

BUILDING BLOCK 3

Spec-Driven Development

The methodology that ties intent, testing, and AI execution together.
Specs replace chat history as the source of truth.

Level 1	**Spec-First**	Write a detailed spec before the agent codes. The right starting point for most teams.
Level 2	**Spec-Anchored**	Spec persists as source of truth. Automated drift detection between spec and code.
Level 3	**Spec-as-Source**	Specification IS the software. Code is continuously regenerated from spec.

ICSE 2026: Architectural docs improve AI code generation measurably in correctness, conformance, and modularity.

PRACTITIONER'S VIEW

Lessons from Peter Steinberger

Creator of OpenClaw | 247K GitHub stars | 300K lines built with AI | Now at OpenAI

Beware the "agentic trap"	Over-optimizing your AI environment makes you feel productive without being productive. Simplicity wins.
Upfront planning matters MORE	"Shape the statue" before the agent carves. High-level architecture by a human, iterative execution by the agent.
Two skills that survive	System architecture and taste — the ability to distinguish between a working solution and an elegant, maintainable one.
The role shift is real	70%+ of AI-using devs now spend more time on problem definition. From coder to orchestrator.

> *Code is a commodity. The soul — intent, architecture, taste — is what has value.*

TOOL 1

GitHub Spec Kit

Enterprise-grade spec-driven development. Backed by GitHub/Microsoft.
Microsoft Learn already has training modules.

Constitution → **Specify** → **Plan** → **Tasks** → **Implement**

Key concept: The Constitution

Non-negotiable project principles that every AI output must respect. Your org's coding standards, security requirements, and architectural patterns codified in a single document the agent always reads.

Best for

- Large teams needing governance
- Compliance-heavy organizations
- Greenfield + brownfield projects
- Enterprise standardization

TOOL 2

OpenSpec

Lightweight, iterative, brownfield-first. YC-backed. 27K+ GitHub stars.
Philosophy: "fluid not rigid, iterative not waterfall."

1 Propose

AI creates change folder:
proposal, specs, design, tasks

→

2 Apply

AI implements tasks.
Specs guide every decision.

→

3 Archive

Specs updated. Change
archived. Ready for next.

Best for: fast-moving teams, existing codebases, low-ceremony environments. 3 commands vs 8. Setup in 2 minutes.

Works with 20+ AI assistants — no vendor lock-in.

COMPARISON

Spec Kit vs OpenSpec

Not a competition — different tools for different contexts within the same org.

	Spec Kit	OpenSpec
Setup time	15-30 minutes	2 minutes
Commands	8 slash commands	3 slash commands
Ceremony	High (constitution, plans, checklists)	Low (propose, apply, archive)
Brownfield	Supported	Brownfield-first
Enterprise governance	Strong (constitution, presets)	Lightweight
Verbosity	~800 lines per feature	~250 lines per feature
Backed by	GitHub / Microsoft	Fission AI (YC)
Best for	Large teams, compliance	Fast teams, existing code

THE FULL PICTURE

Agentic Software Development Lifecycle

Intent
WHY → WHAT

→

Spec
Make it actionable

→

Architecture
Design the system

→

Build + Test
TDD keeps it honest

→

Review
Human checkpoint

→

Deliver
Docs + wiki + tag

Human as team principal → autonomous between checkpoints → collaborative at gates

CASE STUDIES

Enterprise in Practice

SAP / Syntax

The governance warning

VP Matthias Steiner: "Productivity without governance is opening the door to tomorrow's technical debt." Structured AI workflows prevent the mess.

Brownfield .NET CMS

Spec Kit at 307K lines

Extended a legacy CMS with new features using full SDD cycle. Community verdict: "consistency at scale" and "better AI context" across the codebase.

Enterprise at Scale

Faros AI — 10K developers, 1,255 teams

The disconnect: developers say faster, companies don't see velocity gains. Organizations with governance + measurement + change management see real ROI.

What You Already Do → What to Formalize

You write PRDs	→	**Intent-driven development**
You write acceptance criteria	→	**Spec-driven development**
You define "done" before sprints	→	**Test-driven development**
You review at sprint boundaries	→	**Human copilot protocol**

The gap: you're doing it informally, ad hoc, and it's not consumable by AI.
Formalize it and you 10x your output.